

Introduction and Optimization Problems

This opening lecture establishes the course’s central ambition—using computation to understand the world—and introduces optimization problems through their two-part anatomy, their formal notation, and their most famous exemplar, the knapsack problem.

Computational Models: The Computer as Laboratory

The organizing theme of the course is **using computation to help understand the world in which we live**. A *computational model* is an experimental device that helps us either understand something that has happened or predict the future. Where experiments were traditionally performed in a physical laboratory, the shift this course describes is that the computer itself becomes the laboratory: instead of building apparatus, we run experiments in code, asking “what if” questions and rerunning them as many times as we like to learn about the past or the future.

Three flavors of computational models organize the material ahead: **optimization models**, **statistical models**, and **simulation models**. This lecture begins with the first of the three.

Anatomy of an Optimization Model

An optimization model has exactly two ingredients:

1. An **objective function that is to be maximized or minimized** — for example, minimize the time spent traveling from New York to Boston.
2. A **set of constraints — possibly empty — that must be honored** — in the travel example, spend no more than \$100 and be in Boston before 5:00 PM.

So the goal is the fastest trip *subject to* a budget and a deadline. This is precisely the problem that travel sites (TripAdvisor, Kayak, Expedia, and their peers) solve commercially — find the best itinerary subject to the traveler’s constraints — so optimization is embedded in tools used every day, not an abstract exercise.

The formal grounding comes from mathematical optimization: the **selection of a best element, with regard to some criteria, from some set of available alternatives**. The field divides into two subfields depending on whether the variables are **continuous** or **discrete**, and optimization problems arise in all quantitative disciplines, from computer science and engineering to operations research and economics. In the general approach, one maximizes or minimizes a real function by systematically choosing input values from within an allowed set. Typically that allowed set A is a subset of Euclidean space $\mathbb{R}^n$, specified by equalities or inequalities that its members must satisfy. The vocabulary is worth fixing precisely:

- The domain A of the function is called the **search space** or **choice set**; it is the set of all points satisfying the problem’s constraints, targets, or goals. Its size and complexity vary widely — a multidimensional real-valued domain in continuous problems, versus a finite set of permutations, combinations, or configurations in discrete (combinatorial) ones. Navigating it efficiently is crucial, because it directly influences computational complexity and the likelihood of finding an optimal solution.
- Elements of A are called **candidate solutions** or **feasible solutions**.
- The function itself goes by many names across fields: **objective function**, criterion function, **loss** or **cost function** (when minimizing), **utility** or **fitness function** (when maximizing), or **energy function** in physics, where minimization corresponds to the energy of the system being modeled. In machine learning, a cost function continuously evaluates the quality of a data model, where a minimum implies a set of possibly optimal parameters with the lowest error.
- A feasible solution that minimizes (or maximizes) the objective function is the **optimal solution**.

The standard form of a continuous optimization problem is

$$\begin{array}{ll}\underset{x}{\text{minimize}} & f(x) \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p\end{array}$$

If $m = p = 0$, the problem is **unconstrained**. By convention the standard form defines a *minimization* problem; a maximization problem is treated simply by negating the objective function (and the opposite perspective — considering only maximization — would be equally valid).

Historically, Fermat and Lagrange found calculus-based formulae for identifying optima, while Newton and Gauss proposed iterative methods for moving toward an optimum. The term “linear programming” for certain optimization cases is due to George B. Dantzig, though much of the theory was introduced by Leonid Kantorovich in 1939 — and “programming” in that phrase does not refer to computer programming.

Notation: min, max, arg min, arg max

Optimization problems are expressed with special notation, and the distinctions matter:

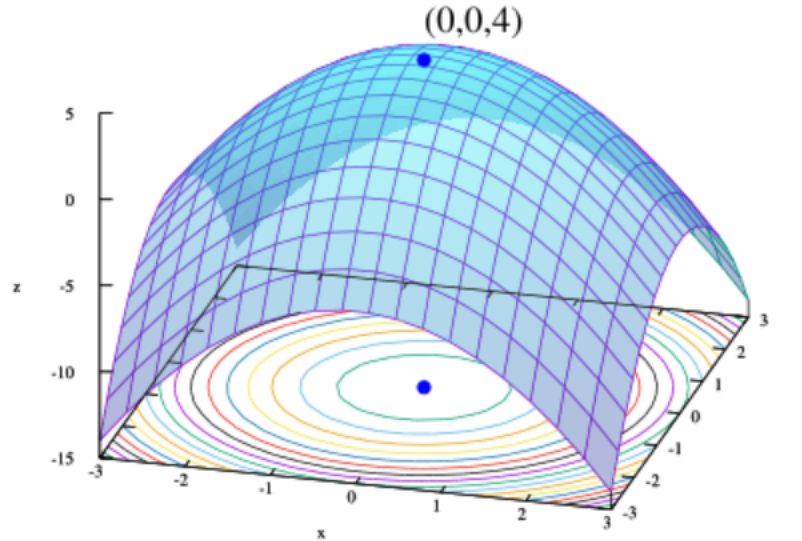
- $\min_{x \in \mathbb{R}} (x^2 + 1)$ asks for the minimum **value** of the objective function: it is 1, occurring at $x = 0$.
- $\max_{x \in \mathbb{R}} (2x)$ has no answer: the objective is unbounded, so the result is “infinity” or “undefined.”
- $\arg \min_{x \in (-\infty, -1]} (x^2 + 1)$ asks instead for the **argument** achieving the minimum, not the value itself. The answer is $x = -1$, because $x = 0$ — where the true minimum sits — is *infeasible*: it does not belong to the feasible set.
- $\arg \max_{x \in [-5, 5], y} x \cos y$ returns the maximizing pairs, which are $\{5, 2k\pi\}$ and $\{-5, (2k + 1)\pi\}$ for all integers k .

The operators $\arg \min$ and $\arg \max$ thus stand for “argument of the minimum” and “argument of the maximum.”

Local versus Global Optima

A **local minimum** x^* is an element for which there exists some $\delta > 0$ such that $f(x^*) \leq f(x)$ holds nearby — that is, on some region around x^* , all function values are greater than or equal to the value at x^* . Local maxima are defined analogously. While a local minimum is at least as good as any *nearby* element, a **global minimum** is at least as good as *every feasible element*. Generally, unless the objective function is convex in a minimization problem, there may be several local minima. In a convex problem, any local minimum that is interior (not on the edge of the feasible set) is also the global minimum; a nonconvex problem, however, may have multiple local minima, not all of which need be global.

This distinction has practical teeth: a large number of algorithms proposed for nonconvex problems — including the majority of commercially available solvers — cannot distinguish locally optimal from globally optimal solutions, and will treat the former as actual solutions to the original problem. The branch of applied mathematics and numerical analysis concerned with deterministic algorithms that *guarantee* convergence in finite time to the true optimum of a nonconvex problem is called **global optimization**.



Source: Wikipedia, article “[Mathematical optimization](#)”.

The Knapsack Problem

The most famous example of an optimization problem is the **knapsack problem**. Picture a burglar’s dilemma: a knapsack that can carry only so much weight, surrounded by valuable items — a Picasso painting, a jeweled crown, a stack of five-hundred-euro notes, a gold bar, an antique wall clock, an Egyptian-style bust. Each item has both a weight and a value; which items do you take to maximize total value without breaking the knapsack? The structure maps exactly onto the definition above: an objective function (maximize value) plus a constraint (the weight limit). The burglar story sounds silly, but the abstraction appears everywhere, and subsequent lectures tackle how to solve it computationally.

Formally, the knapsack problem is a problem in **combinatorial optimization**, named for someone constrained by a fixed-size knapsack who must fill it with the most valuable items. It typically arises in **resource allocation**, where decision-makers choose from a set of non-divisible projects or tasks under a fixed budget or time constraint. It has been studied for more than a century, with early work dating back to 1897.

Its principal variants differ in how many copies of each item may be taken. Given n items numbered 1 to n , each with weight w_i and value v_i , and a maximum weight capacity W , with x_i the number of instances of item i included:

- The **0-1 knapsack problem** — the most common — restricts each x_i to zero or one:

$$\max \sum_{i=1}^n v_i x_i \quad \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

- The **bounded knapsack problem (BKP)** allows up to some maximum non-negative integer c copies of each kind of item.
- The **unbounded knapsack problem (UKP)** places no upper bound: x_i may be any non-negative integer.

A special case is the **subset sum problem**, where for each item the weight equals the value ($w_i = v_i$); in cryptography, “knapsack problem” often refers specifically to subset sum, which is one of Karp’s 21 NP-complete problems.

Real-world applications span finding the least wasteful way to cut raw materials, selection of investments and portfolios, selection of assets for asset-backed securitization, and generating keys for the Merkle–Hellman and other knapsack cryptosystems. One early application was constructing and scoring tests where test-takers choose which questions to answer: Feuerman and Weiss proposed giving students a heterogeneous test with 125 possible points, and having a knapsack algorithm determine, among the subsets of questions whose point values add up to 100, which subset yields each student the highest possible score. A 1999 study of the Stony Brook University Algorithm Repository ranked the knapsack problem 19th most popular — and third most needed, after suffix trees and bin packing — among 75 algorithmic problems in combinatorial algorithms and algorithm engineering.

Like every optimization problem, the knapsack has a corresponding **decision problem** asking whether a feasible solution exists for some particular measure — generically, if an optimization problem is “find a path from u to v using the fewest edges” (with answer, say, 4), the decision version asks “is there a path from u to v using 10 or fewer edges?”, answerable with a simple yes or no. For the knapsack, the two versions are of similar difficulty: a polynomial-time algorithm for the decision problem lets you find

The Knapsack Problem: A First Optimization Problem

The knapsack problem is a classic problem in **combinatorial optimization**: someone constrained by a fixed-size knapsack must fill it with the most valuable items. The story is intuitive — you’re going on an expedition, your strength limits you to a maximum weight you can carry, and yet you’d like to take more stuff than you can carry. The question becomes: which items do you take, and which do you leave behind?

This structure appears everywhere once you start looking for it. A personal example: a table laden with a hot dog, spaghetti and meatballs, a salad, gingerbread cookies, an ice cream sundae, and a sandwich — where the “knapsack” is your stomach. There’s only so much you can eat; you’d like to eat all of it; so which items do you take? Beyond such everyday examples, knapsack problems arise in real-world resource allocation wherever decision-makers must choose from a set of non-divisible projects or tasks under a fixed budget or time constraint. Documented applications include finding the least wasteful way to cut raw materials, selecting investments and portfolios, selecting assets for asset-backed securitization, generating keys for the Merkle–Hellman knapsack cryptosystem, and even constructing exams where test-takers choose which questions to answer (a knapsack algorithm can pick, from subsets of questions totaling 100 points, the subset giving each student the highest score). The problem has been studied for over a century, with early work dating back to 1897, and a 1999 survey of the Stony Brook Algorithm Repository ranked it among the most popular algorithmic problems — 19th out of 75, and third most needed after suffix trees and bin packing.

Two Variants: 0/1 versus Fractional

It’s essential to keep two variants distinct:

- **0/1 knapsack problem**: each item is either taken or not taken — binary, one or zero, in or out. If your loot is a gold bar, you can’t saw off a piece; you take the whole bar or leave it behind.
- **Continuous (fractional) knapsack problem**: you may take any fraction of an item. If your loot is a pile of gold dust, you scoop up as much or as little as you like.

The lecture focuses on the 0/1 version because it turns out to be the more interesting and more common case. Wikipedia’s taxonomy confirms this framing: the 0-1 knapsack problem restricts the number x_i of copies of each item to zero or one; the *bounded* knapsack problem relaxes this to allow up to some maximum integer count c of copies per item; and the *unbounded* knapsack problem places no upper bound on copies at all, requiring only that each x_i be a non-negative integer.

A notable special case is the **subset sum problem**, where for each item the weight equals the value ($w_i = v_i$).

In cryptography, “knapsack problem” often refers specifically to subset sum, which is one of Karp’s 21 NP-complete problems — a hint of the computational difficulty lurking beneath this innocent-looking question.

Formalizing 0/1 Knapsack

To write programs that solve the problem, we state it precisely.

Setup. Each item is represented by a pair: a *value* and a *weight*. Every item knows how much it’s worth to you and how much it costs you to carry. The knapsack can accommodate items with total weight no more than w — our constraint, our limited strength.

Representation. We use two vectors, both of length n :

- L , the vector of available items, one element per item.
- V , the **decision vector** indicating whether items are taken:

$$V[i] = \begin{cases} 1 & \text{if item } L[i] \text{ is taken} \\ 0 & \text{otherwise} \end{cases}$$

A solution to the problem is simply a choice of V : a bunch of ones and zeros saying “take this, leave that.”

Objective and constraint. We want to find a V that maximizes total value,

$$\max_V \sum_{i=0}^{n-1} V[i] \cdot L[i].\text{value}$$

subject to the weight constraint

$$\sum_{i=0}^{n-1} V[i] \cdot L[i].\text{weight} \leq w.$$

Notice how the same trick serves both sums: multiplying each item’s attribute by whether we took it means untaken items ($V[i] = 0$) contribute nothing, while taken items contribute their full value or weight. So the first sum is exactly the total value of everything in the knapsack, and the second is exactly the total weight carried. The whole problem, stated precisely: **maximize total value without exceeding the weight limit.**

Brute Force: The Obvious First Attack

The natural first thing to try is a **brute force algorithm**, in three steps:

1. **Enumerate all possible combinations of items** — generate every possible way of choosing which things to take.
2. **Remove all combinations whose total weight exceeds the allowed weight** — throw out the illegal choices, the ones where we’ve overpacked and can’t lift the knapsack.
3. **From the remaining combinations, choose any one whose value is largest.**

This will work and is guaranteed to give the optimal answer, because we’ve literally looked at everything. In computer science terms, brute-force search (also called exhaustive search or “generate and test”) systematically checks all possible candidates against the problem’s statement. It is simple to implement and always finds a solution if one exists — which is why it’s the right place to start: it gives us a correct baseline against which everything cleverer can be judged. Brute force is also valuable when simplicity matters more than speed (e.g., critical applications where algorithmic errors would have serious consequences), and it serves as the standard baseline method when benchmarking other algorithms and metaheuristics.

But we must think about the **cost**. How many subsets does a set of n items have? On the order of 2^n . And that grows very, very fast — which will motivate looking for something cleverer.

Power Sets: Counting All Subsets

Step one of brute force generates every subset of the item set. This collection has a name: the **power set**.

In mathematics, the power set $\mathcal{P}(S)$ of a set S is the set of *all* subsets of S , including the empty set and S itself. For example, if $S = \{x, y, z\}$, then

$$\mathcal{P}(S) = \{\{\}, \{x\}, \{y\}, \{z\}, \{x, y\}, \{x, z\}, \{y, z\}, \{x, y, z\}\}.$$

The key fact for us: if S is finite with $|S| = n$, then the number of subsets is

$$|\mathcal{P}(S)| = 2^n.$$

There's a beautiful reason for this, visible in the notation 2^S : since 2 can be defined as the set $\{0, 1\}$, the expression 2^S denotes the set of all functions from S to $\{0, 1\}$ — and a function assigning each element of S a 0 or a 1 is *exactly* a decision vector V saying “in or out.” Each subset corresponds to one binary string of length n , and there are 2^n such strings. Equivalently, via the binomial theorem, the number of k -element subsets is the binomial coefficient $\binom{n}{k}$, and summing over all sizes recovers the identity

$$2^n = \sum_{k=0}^n \binom{n}{k}.$$

So enumerating the power set means enumerating 2^n candidate solutions — one per possible decision vector.

Combinatorial Explosion: Why Brute Force Doesn't Scale

The steep growth in the number of candidates as data size increases is called the **combinatorial explosion** (or the curse of dimensionality), and it afflicts brute-force methods across all sorts of problems, not just knapsack.

Concrete illustrations of how punishing exponential growth is:

- Searching for divisors of a number n by testing every integer up to n : if n has sixteen decimal digits, the search requires at least 10^{15} computer instructions — several days on a typical PC. A random 64-bit number averages about 19 decimal digits, pushing the search to roughly 10 years.
- Seeking a particular rearrangement of letters: 10 letters give $10! = 3,628,800$ candidates, generatable and testable in under a second. Adding just one more letter — a mere 10% increase in data size — multiplies the candidates by 11 (a 1000% increase). At 20 letters, $20! \approx 2.4 \times 10^{18}$ candidates means about 10 years of search.
- Chess endgame tablebases: all endings with six pieces or fewer were solved by 2005; completing the seven-piece tablebase took ten more years; adding one more piece (an eight-piece tablebase) is considered intractable due to added combinatorial complexity.

For knapsack specifically, the candidate count is 2^n subsets, so doubling the number of items squares the work. This is why brute force is typically used only when problem size is limited, or when problem-specific heuristics can shrink the candidate set to something manageable. One classic refinement worth distinguishing: **backtracking**, which discards large sets of solutions *without explicitly enumerating them* — unlike pure brute force, which must generate every candidate before testing it.

The deeper lesson about knapsack's difficulty comes from complexity theory: its hardness depends on the form of the input. With integer weights and profits it is **weakly NP-complete**; with rational weights and profits it is **strongly NP-complete** (though it still admits a fully polynomial-time approximation scheme in that case). The NP-hardness relates to computational models where the size of integers matters, such as Turing machines. Research literature also studies what makes particular instances “hard” — partly to build public-key cryptosystems like Merkle-Hellman on hard instances, and partly because understanding the

structure of an optimization problem’s instance space improves algorithm selection. For now, the takeaway is clear: exhaustive enumeration is correct but exponentially expensive, and we’ll need smarter ideas.

MIT OpenCourseWare: where every piece of this course lives

This course is published through **MIT OpenCourseWare** (OCW), an initiative of MIT to publish all of the educational materials from its undergraduate- and graduate-level courses online, freely and openly available to anyone, anywhere. Announced on April 4, 2001, the program releases its materials under a Creative Commons Attribution-NonCommercial-ShareAlike license. That license is the practical reason the closing slide points to the Terms of Use page: copyright in OCW material remains with MIT, its faculty, or its students, so anyone citing the materials in a report or reusing them in their own teaching must do so under the stated terms rather than by default assumption.

The idea emerged in 1999 from the MIT Council on Education Technology, which provost Robert Brown charged with determining how MIT should position itself in the distance-learning/e-learning environment. OCW was conceived as a new model for the dissemination of knowledge and collaboration among scholars worldwide — a contribution to the “shared intellectual commons” of academia — spearheaded by professors Dick K.P. Yue, Shigeru Miyagawa, and Hal Abelson. Notably, the main implementation challenge was *not* faculty resistance but logistics: determining ownership and obtaining publication permission for the massive amount of copyrighted items embedded in faculty course materials, plus the time and technical effort required to convert everything to an online format.

The project scaled quickly. A proof-of-concept pilot opened in September 2002 with 32 courses; the 500th course appeared in September 2003 (some already with complete streaming video lectures); 900 courses were online by September 2004; and over 2,400 courses were available as of May 2018. A majority provide homework problems and exams (often with solutions) and lecture notes; some add interactive web demonstrations in Java, complete textbooks written by MIT professors, and streaming video — 100 courses included complete video lectures as of May 2018, streamable or downloadable, and mirrored on YouTube, iTunes U, and the Internet Archive. By 2020 the platform reported materials accessed by over 500 million learners worldwide since inception. For this course, that means the slides, recordings, and supporting materials remain permanently retrievable at ocw.mit.edu.

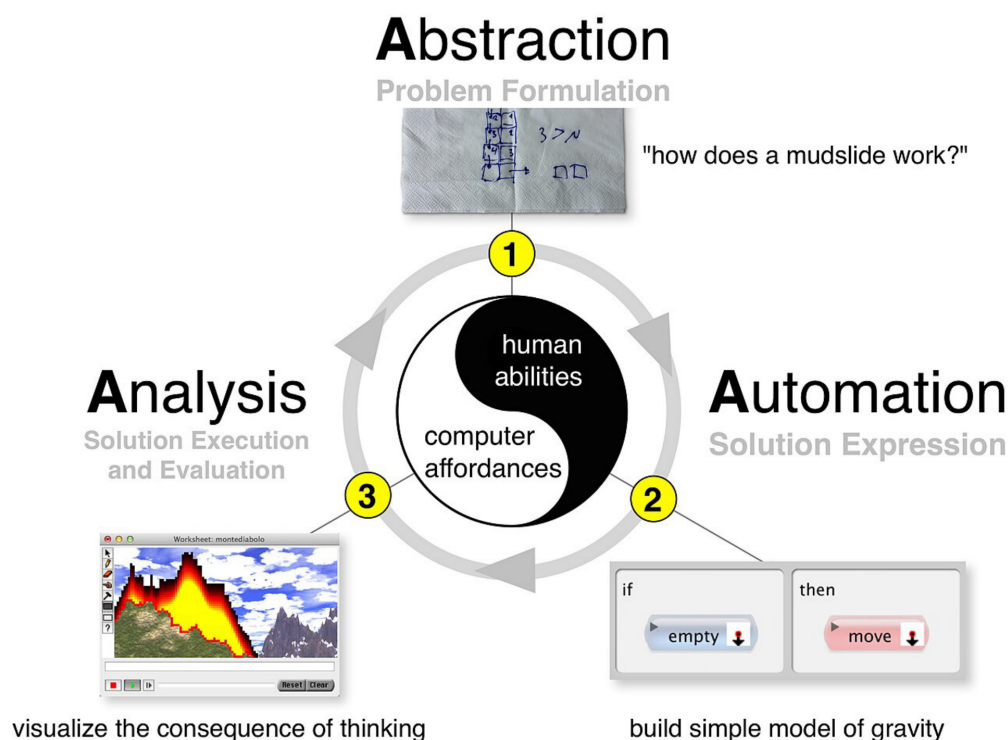
OCW also reshaped the wider landscape. It inspired other institutions to release open educational resources, and in 2005 it joined other projects in forming the OpenCourseWare Consortium, which seeks to extend the reach and impact of open course materials, foster new ones, and develop sustainable publication models. In 2007 it introduced *Highlights for High School*, indexing resources applicable to advanced high-school study in biology, chemistry, calculus, and physics to support US STEM education at the secondary level. In 2011 came the first of fifteen *OCW Scholar* courses, designed specifically for independent learners: more in-depth than standard publications, with materials sequenced logically to facilitate self-study (no on-site interaction is supported, though study groups on the collaborating project OpenStudy exist for some courses). In 2012, Harvard and MIT launched edX as a MOOC provider, and between 2013 and 2019 some OCW courses were delivered through the European MOOC platform Eliademy.

Operationally, OCW was originally funded by the William and Flora Hewlett Foundation, the Andrew W. Mellon Foundation, and MIT, and is now supported by MIT, corporate underwriting, major gifts, and donations from site visitors; as of 2013 it cost about US\$3.5 million per year (roughly \$4.8 million in 2025 dollars), with a stated 2011 goal of increasing reach ten-fold over the following decade. Technically, it began on a custom content management system built on Microsoft’s Content Management Server, replaced in mid-2010 by a Plone-based system; MIT describes the whole apparatus as a “large-scale digital publishing infrastructure” of planning tools, a CMS, and a content-distribution layer. Course video was originally primarily in RealMedia format; in 2008 YouTube became the primary streaming platform, with videos embedded back into the OCW site, while full files remained downloadable from iTunes U and the Internet Archive. A 2011 iPhone app called LectureHall (with Irynsoft) and the MIT SOUL system for video processing round out the delivery technology.

Computational thinking: the discipline this course has been practicing

Computational thinking (CT) refers to the thought processes involved in formulating problems so their solutions can be represented as computational steps and algorithms. In education, it is a set of problem-solving methods that involve expressing problems and their solutions in ways a computer could also execute. It encompasses both the automation of processes and the use of computing to explore, analyze, and understand processes — natural and artificial alike. As the umbrella skill of a course titled *Introduction to Computational Thinking and Data Science*, it is what turns an unstructured real-world question into something an algorithm can act on.

Four characteristics define CT: **decomposition**, **pattern recognition/data representation**, **generalization/abstraction**, and **algorithms**. The mechanism connecting them explains why the approach is powerful: by decomposing a problem, identifying the variables involved through data representation, and creating algorithms, one arrives at a *generic* solution — a generalization or abstraction that solves a multitude of variations of the initial problem. This is why CT can be used to algorithmically solve complicated problems of scale and often realizes large improvements in efficiency: solving the abstracted problem once beats solving each variant separately. An equivalent characterization is the “three As” iterative process — abstraction, automation, analysis — shown below.



Source: Wikipedia, article "[Computational thinking](#)".

The underlying ideas are old: the concept dates back at least to the 1950s, with most ideas far older, and was preceded by terms like *algorithmizing*, *procedural thinking*, *algorithmic thinking*, and *computational literacy* from computing pioneers such as Alan Perlis and Donald Knuth. Seymour Papert used the term “computational thinking” in 1980 and again in 1996. What brought it to the forefront of computer science education was Jeannette Wing’s 2006 essay in *Communications of the ACM*, which argued that thinking computationally is a fundamental skill for *everyone*, not just computer scientists, and urged integrating computational ideas into other school subjects — even claiming children would improve at everyday tasks like packing a backpack, finding lost mittens, and knowing when to stop renting and buy instead. CT shares its core moves (abstraction, data representation, logically organizing data) with scientific, engineering, systems,

design, and model-based thinking — a kinship that, as noted below, later made drawing its boundaries difficult.

Within education, CT has been positioned alongside the four Cs of 21st-century learning (communication, critical thinking, collaboration, creativity) as a possible fifth C: the capability to resolve problems algorithmically and logically, including tools that produce models and visualize data. Grover describes its applicability beyond STEM, into the social sciences and language arts, and the “algoRithms” component has been called the “fourth R” alongside Reading, wRiting, and aRithmetic. Like Papert, Perlis, and Marvin Minsky before her, Wing envisioned CT becoming an essential part of every child’s education — and adoption has